PRACTICAL HANDBOOK

GenAI Evaluation

A practical guide to evaluating LLMs, RAG systems, agents, and production AI applications

The goal

Move from “the response looks good” to a repeatable evaluation system with measurable quality, safety, cost, and reliability.

What this guide covers

- **LLM Evaluation -** correctness, relevance, hallucination, coherence, safety, and instruction following.
- **RAG Evaluation -** retrieval quality plus grounded answer quality.
- **Agent Evaluation -** task success, tool use, planning, efficiency, cost, and latency.
- **Production Evaluation -** latency, token usage, cost, reliability, safety, and user feedback.
- **Practical implementation -** test datasets, graders, scorecards, thresholds, and release gates.

Best use: classroom notes, project documentation, interview preparation, and production evaluation design.

1. Evaluation Fundamentals

Understand what should be evaluated before choosing a metric or framework

1.1 What is GenAI evaluation?

GenAI evaluation is the systematic process of measuring whether an AI system produces the desired result, uses the right evidence or tools, follows constraints, remains safe, and performs efficiently enough for the target application.

> **Important principle**
> Evaluate the complete system, not only the language model. A high-quality model can still produce a poor application if retrieval is weak, tools fail, prompts are wrong, or production latency is too high.

1.2 Four evaluation layers

Layer	Main question	Typical metrics
LLM	Is the generated response good?	Correctness, relevance, hallucination, coherence, safety, instruction following
RAG	Did we retrieve the right evidence and generate from it?	Precision, recall, hit rate, MRR, NDCG, faithfulness, answer relevance
Agent	Did the agent complete the task correctly and efficiently?	Task success, tool selection, tool call accuracy, planning quality, steps, cost, latency
Production	Does the application perform reliably for real users?	Latency, token usage, cost, reliability, safety, user feedback

1.3 A running example used in this guide

Imagine a company has an employee policy assistant. A user asks:

> **User question**
> “How many paid parental leave days do employees get, and does it apply during probation?”

The application may need to retrieve the leave policy, identify the relevant paragraph, generate a factual answer, and possibly call an HR system if eligibility depends on employee status. This single example lets us evaluate LLM quality, RAG retrieval, agent behavior, and production performance separately.

1.4 Evaluation workflow

1. Define the task and expected behavior.
2. Create representative test cases, including normal and difficult cases.
3. Choose metrics for each system component.
4. Run the system and collect outputs, retrieved context, tool calls, latency, and token usage.
5. Score using deterministic checks, human review, or model-based graders.
6. Set thresholds and compare against a baseline.
7. Investigate failures by component instead of relying only on one overall score.

2. LLM Evaluation

Evaluate the quality of the generated response itself

LLM evaluation focuses on the final generated text. The same response can score high on one dimension and low on another. For example, a response may be fluent and coherent but factually wrong.

2.1 Accuracy / Correctness

Core question	Is the answer factually or logically correct?
What it measures	Measures whether the generated answer matches known facts, a reference answer, or task-specific truth.
Example	Reference: “20 paid working days.” Model: “20 paid working days.” -> correct. Model: “30 calendar days.” -> incorrect.
How to interpret	High correctness means the answer is materially right. For open-ended tasks, use semantic or rubric-based grading rather than exact string matching.
Typical evaluation method	Reference comparison, unit tests, domain rules, or LLM-as-a-judge with a rubric.

2.2 Relevance

Core question	Does the answer directly address the user’s request?
What it measures	Measures whether the response stays focused on the requested information rather than adding unrelated content.
Example	User asks only about parental leave. A response spends half its text explaining annual leave -> lower relevance.
How to interpret	A response can be correct but still poorly relevant if it is verbose, tangential, or misses the main intent.
Typical evaluation method	Human/LLM rubric, keyword coverage, semantic similarity, task-specific checks.

2.3 Hallucination

Core question	Did the model invent unsupported facts?
What it measures	Measures factual claims that are not supported by known truth or supplied evidence.

Example	Retrieved policy says probation eligibility is not specified. Model says “employees on probation are not eligible.” -> hallucination unless another valid source supports it.
How to interpret	Lower hallucination is better. In RAG systems, hallucination is often operationalized through faithfulness to retrieved context.
Typical evaluation method	Claim extraction + evidence verification, human review, LLM judge, groundedness checks.

2.4 Coherence

Core question	Is the response logically connected and easy to follow?
What it measures	Measures consistency, readability, flow, and whether statements contradict each other.
Example	Bad: “Employees receive 20 days. Therefore, there is no paid leave.” The statements conflict.
How to interpret	Coherence is about structure and internal logic, not external factual correctness.
Typical evaluation method	Human/LLM rubric; language-quality checks.

2.5 Toxicity / Safety

Core question	Does the model avoid harmful, toxic, disallowed, or privacy-violating behavior?
What it measures	Measures compliance with safety requirements and domain-specific policies.
Example	A support assistant should not expose another employee’s confidential medical details or generate abusive content.
How to interpret	Safety should often be treated as a release gate, not merely averaged with quality scores.
Typical evaluation method	Safety classifiers, policy-based graders, adversarial test sets, human review.

2.6 Instruction Following

Core question	Did the model obey explicit constraints?
What it measures	Measures whether the response follows format, tone, length, schema, tool-use, and content instructions.

Example	Instruction: “Return JSON with keys answer and source.” Valid JSON with both keys -> pass. Free-form prose -> fail.
How to interpret	This metric is especially useful when outputs feed another program.
Typical evaluation method	JSON/schema validation, regex, exact constraints, or rubric-based grading.

2.7 One response, multiple scores

Response characteristic	Possible result
Correct facts but very long and off-topic	High correctness, lower relevance
Fluent and well structured but invented policy detail	High coherence, low groundedness / high hallucination
Correct content but wrong JSON schema	High correctness, low instruction following
Safe refusal to a disallowed request	High safety even if normal task-completion score is low

3. RAG Evaluation - Retrieval

Measure whether the retriever finds the right evidence before generation

A RAG system has at least two separate quality problems: retrieval and generation. If the correct document never enters the context window, even a strong LLM may fail. Therefore, evaluate retrieval independently.

Example retrieval setup

For a query, assume the gold-standard relevant chunks are {A, C, F, H}. The retriever returns top-5 chunks in this order: [B, A, D, C, F]. Three of the five retrieved chunks are relevant.

3.1 Precision@k

Precision@k = Relevant retrieved items in top-k / k

Precision asks: "Of what I retrieved, how much was actually useful?"

Example: 3 relevant chunks in top-5 -> Precision@5 = 3/5 = 0.60**Example: 3 relevant chunks in top-5 -> Precision@5 = 3/5 = 0.60**

Use precision when irrelevant context is expensive because it consumes tokens, distracts the LLM, or increases latency.

3.2 Recall@k

Recall@k = Relevant retrieved items in top-k / Total relevant items

Recall asks: "Of all useful evidence available, how much did I find?"

Example: retrieved 3 of 4 relevant chunks -> Recall@5 = 3/4 = 0.75**Example: retrieved 3 of 4 relevant chunks -> Recall@5 = 3/4 = 0.75**

Use recall when missing a critical piece of evidence is risky. Increasing k often improves recall but may reduce precision.

3.3 Hit Rate@k

Hit Rate@k = 1 if at least one relevant item appears in top-k, otherwise 0

Across a dataset, average the binary result over all queries.

Example query has relevant chunks in top-5 -> Hit@5 = 1**Example query has relevant chunks in top-5 -> Hit@5 = 1**

Hit Rate is simple and useful for "did we find anything relevant?" tasks, but it does not tell you how many relevant items were found.

3.4 Mean Reciprocal Rank (MRR)

RR = 1 / rank of the first relevant result; MRR = mean(RR over all queries)

MRR rewards systems that place the first useful result near the top.

If first relevant result is at rank 2 -> RR = 1/2 = 0.50**If first relevant result is at rank 2 -> RR = 1/2 = 0.50**

MRR is especially useful when one highly relevant result is enough, such as FAQ retrieval or answer passage lookup.

3.5 NDCG@k

NDCG@k = DCG@k / IDCG@k

NDCG considers both ranking position and graded relevance. A highly relevant result at rank 1 should receive more credit than the same result at rank 5.

Unlike binary metrics, NDCG is useful when results can be “highly relevant”, “partially relevant”, or “not relevant”. Scores are typically normalized between 0 and 1, where higher is better.

Metric	Best question it answers	Good fit
Precision@k	How clean is the retrieved context?	Reduce noise / token waste
Recall@k	Did we find all important evidence?	Compliance, research, multi-fact answers
Hit Rate@k	Did we retrieve at least one useful result?	Simple QA / lookup
MRR	How early is the first useful result?	FAQ, passage lookup
NDCG@k	Are the most relevant results ranked highest?	Search with graded relevance

4. RAG Evaluation - Generation

Measure whether the answer uses retrieved context correctly

After retrieval, evaluate the generated answer. RAG generation metrics focus on grounding and usefulness. These metrics are related but not interchangeable.

4.1 Faithfulness

Core question	Are the answer's claims supported by the retrieved context?
What it measures	Measures whether the response stays grounded in the provided evidence.
Example	Context: "Employees receive 20 paid working days." Answer: "Employees receive 20 paid working days." -> faithful. Adding an unsupported probation rule -> less faithful.
How to interpret	Faithfulness does not require the answer to be complete; it requires claims that are made to be supported.
Typical evaluation method	Claim-by-claim support check, groundedness grader, human review.

4.2 Answer Relevance

Core question	Does the final answer satisfy the user's question?
What it measures	Measures how directly and completely the answer addresses the query.
Example	If the user asks about duration and probation but the answer gives only duration, relevance/completeness is reduced.
How to interpret	A response can be perfectly faithful to context but still fail to answer the actual question.
Typical evaluation method	Question-answer semantic/rubric grading.

4.3 Context Relevance

Core question	Was the retrieved context actually useful for answering the question?

What it measures	Measures how much of the supplied context is pertinent rather than distracting or unrelated.
Example	Five retrieved chunks include two parental-leave chunks and three cafeteria-policy chunks -> context relevance is poor.
How to interpret	This bridges retrieval and generation: noisy context can lower answer quality even when a relevant chunk is present.
Typical evaluation method	Chunk relevance labels, LLM/context grader, retrieval annotations.

4.4 Answer Correctness

Core question	Is the generated answer actually right?
What it measures	Measures agreement with ground truth or expected facts, often combining factual and semantic correctness.
Example	Gold answer: “20 paid working days; policy does not specify probation.” Generated answer matches both -> high correctness.
How to interpret	Correctness can be low even if the answer is faithful to an outdated or incomplete retrieved source. That is why source quality matters.
Typical evaluation method	Reference-based grading, human SME review, LLM judge with reference.

4.5 Why these four metrics must be separated

Situation	Faithfulness	Answer relevance	Answer correctness
Answer copies context accurately, but context is outdated	High	Possibly high	Low
Answer is supported but only answers half the question	High	Low/medium	Partial
Answer is correct from model memory but unsupported by retrieved context	Low	High	High
Answer uses noisy context but selects the right evidence	High	High	High

5. RAG Worked Example

A complete example from retrieval labels to answer-level scoring

5.1 Query and gold evidence

Query

"How many paid parental leave days do employees receive, and is probation eligibility stated in the policy?"

- Gold chunk A: Employees are eligible for 20 paid working days of parental leave.
- Gold chunk C: The policy section does not specify a separate probation rule.
- Other chunks: annual leave, cafeteria policy, holiday calendar.

5.2 Retriever output

Rank	Chunk	Relevant?	Notes
1	Annual leave policy	No	Noise
2	Parental leave duration	Yes	Critical evidence
3	Probation FAQ	Yes	Useful evidence
4	Holiday calendar	No	Noise
5	Benefits summary	Partially	Related but weaker

- Precision@5: if only ranks 2 and 3 are labeled relevant -> 2/5 = 0.40.
- Hit@5 = 1 because at least one relevant item was retrieved.
- Reciprocal Rank = 1/2 = 0.50 because the first relevant result is at rank 2.
- Recall requires knowing the total number of relevant chunks in the corpus.

5.3 Generated answer

Model answer

"Employees receive 20 paid working days of parental leave. The supplied policy does not state a separate probation restriction, so eligibility during probation should be confirmed with HR if another policy governs it."

Dimension	Assessment	Reason
Faithfulness	High	Claims are supported and uncertainty is preserved.

Answer relevance	High	Answers both duration and probation aspects.
Context relevance	Medium	Retriever included some unrelated chunks.
Answer correctness	High	Matches the gold answer.

6. Agent Evaluation

Evaluate whether an AI agent chooses actions and tools correctly

Agent evaluation is broader than text quality. An agent may produce a polished final response while taking the wrong actions, calling unsafe tools, using incorrect parameters, or wasting many steps. Therefore, evaluate both outcome and trajectory.

6.1 Running agent example

User task

"Check my parental leave balance and create an HR ticket if my profile is missing eligibility information."

Available tools: get_employee_profile(employee_id), get_leave_policy(policy_name), get_leave_balance(employee_id), create_hr_ticket(employee_id, reason).

6.2 Task Success

Core question	Did the agent complete the requested objective?
What it measures	Measures end-to-end outcome rather than individual actions.
Example	Agent checks profile, confirms missing eligibility field, retrieves balance, and creates the requested HR ticket -> success.
How to interpret	Often the most important agent metric. Define success criteria before running the agent.
Typical evaluation method	Deterministic end-state checks, database state, simulator, human/LLM rubric.

6.3 Tool Selection

Core question	Did the agent choose the correct tool for the current subtask?
What it measures	Measures whether the agent selected an appropriate action from available tools.
Example	To check leave balance, using get_leave_balance is correct; using create_hr_ticket first is premature.

How to interpret	Tool choice can be scored per step or per complete trajectory.
Typical evaluation method	Expected-tool labels, trajectory graders, rules.

6.4 Tool Call Accuracy

Core question	Were tool arguments and invocation details correct?
What it measures	Measures parameter accuracy, schema compliance, and whether calls can execute correctly.
Example	get_leave_balance(employee_id="E102") -> correct. employee_id="leave-policy" -> incorrect argument.
How to interpret	Separate selection from argument correctness: the right tool with wrong parameters is still a failure.
Typical evaluation method	Schema validation, exact argument checks, mocked tool execution.

6.5 Planning Quality

Core question	Did the agent choose a sensible sequence of actions?
What it measures	Measures decomposition, dependencies, ordering, and whether unnecessary branches were avoided.
Example	Correct: inspect profile -> check balance -> create ticket only if field is missing. Poor: create ticket before checking profile.
How to interpret	Planning quality is useful when multiple valid paths exist and simple exact-match trajectories are too rigid.
Typical evaluation method	Rubric-based trajectory judge, graph constraints, human review.

6.6 Number of Steps

Core question	How efficiently did the agent reach the goal?
What it measures	Counts tool calls, reasoning/action iterations, or workflow nodes.
Example	Expected 3 calls; agent uses 9 repeated calls -> inefficient.
How to interpret	Fewer is not always better. Optimize steps only after preserving task success and safety.
Typical evaluation method	Trace instrumentation and baseline comparison.

6.7 Cost

Core question	How much did one successful task cost?
What it measures	Measures model tokens, tool/API charges, retrieval cost, and infrastructure cost per task.
Example	Agent A costs $0.04/task, Agent B $0.01/task with similar success -> B may be preferable.
How to interpret	Compare cost together with success and quality; a cheap failing agent is not efficient.
Typical evaluation method	Token accounting + API/tool billing + infra allocation.

6.8 Latency

Core question	How long does the agent take to finish?
What it measures	Measures end-to-end time and sometimes per-tool/model latency.
Example	Three sequential external tools may produce 8-second latency even if the LLM itself responds in 1 second.
How to interpret	Track percentile latency because averages hide slow-tail behavior.
Typical evaluation method	Tracing, p50/p95/p99 latency, per-step timings.

7. Agent Trajectory Example

Inspect actions step by step, not only the final answer

Step	Agent action	Expected?	Evaluation
1	get_employee_profile(E102)	Yes	Correct tool and argument
2	get_leave_balance(E102)	Yes	Correct tool and argument
3	get_leave_policy("parental_leave")	Acceptable	Useful evidence before ticket
4	create_hr_ticket(E102, "eligibility field missing")	Yes	Condition satisfied; correct side effect
5	Final response with balance + ticket ID	Yes	Task completed

7.1 Example agent scorecard

Metric	Score	Threshold	Result
Task success	1.00	>= 0.95	PASS
Tool selection accuracy	1.00	>= 0.95	PASS
Tool argument accuracy	1.00	>= 0.98	PASS
Planning quality	4.5 / 5	>= 4.0	PASS
Mean tool calls	4	<= 5	PASS
p95 latency	6.8 s	<= 8 s	PASS
Cost / task	$0.018	<= $0.025	PASS

Do not over-optimize one metric

Reducing tool calls from 4 to 2 is not an improvement if task success drops. Agent evaluation should use a hierarchy: safety and correctness first, efficiency second.

8. System / Production Evaluation

Measure whether the real application is fast, reliable, affordable, and safe

Offline test-set scores are necessary but not sufficient. Production evaluation measures real system behavior under real traffic, real user diversity, network variation, tool failures, and changing data.

8.1 Latency

Core question	How quickly does the application respond?
What it measures	Measures response time end-to-end or by component.
Example	RAG request: retrieval 250 ms + reranking 400 ms + LLM 2.4 s + network 300 ms -> about 3.35 s total.
How to interpret	Track p50, p95, and p99. p95 means 95% of requests are at or below that latency.
Typical evaluation method	APM/tracing, server logs, model gateway telemetry.

8.2 Token Usage

Core question	How many input and output tokens are consumed?
What it measures	Measures prompt/context/output volume and helps explain cost and latency.
Example	A retrieval change increases average prompt tokens from 4k to 12k without quality gain -> likely inefficient.
How to interpret	Track by endpoint, user flow, model, and document size.
Typical evaluation method	LLM usage metadata and tracing.

8.3 Cost

Core question	What does each request, successful task, or user session cost?
What it measures	Combines model, embedding, reranking, vector DB, tool/API, and infrastructure cost.
Example	A workflow costs $0.03 per request; at 1M requests/month, variable AI cost is about $30k before other infrastructure.
How to interpret	Use cost per successful task, not only cost per raw request.

Typical evaluation method	Billing exports + application telemetry.

8.4 Reliability

Core question	Does the system consistently complete requests without technical failure?
What it measures	Measures availability, error rate, timeout rate, retry rate, and successful completion.
Example	99.9% API uptime can still hide a 7% tool-call failure rate in one agent workflow.
How to interpret	Define reliability at the user-task level as well as service level.
Typical evaluation method	Error monitoring, traces, synthetic tests, SLOs.

8.5 Safety

Core question	Does the deployed system remain within policy under real and adversarial usage?
What it measures	Measures policy violations, leakage, harmful outputs, prompt injection resilience, and unsafe tool actions.
Example	A RAG prompt injection inside a document attempts to make the agent reveal secrets. The system should ignore or contain the attack.
How to interpret	Production safety requires monitoring plus pre-release red-team test cases.
Typical evaluation method	Safety graders, content filters, policy logs, adversarial suites.

8.6 User Feedback

Core question	Do real users find the output useful?
What it measures	Measures direct and indirect satisfaction signals.
Example	Thumbs-up/down, “resolved my issue”, correction rate, retry/rephrase rate, escalation to human support.
How to interpret	User feedback can reveal failures that offline benchmarks do not represent.
Typical evaluation method	Explicit ratings, implicit behavioral signals, support outcomes.

9. How to Score: Evaluation Methods

Choose the grader based on how objectively the requirement can be checked

Method	How it works	Best for	Limitation
Deterministic / rule-based	Code checks exact behavior	JSON schema, tool args, regex, exact values, unit tests	Weak for open-ended quality
Reference-based	Compare with expected answer	QA, extraction, known ground truth	Needs high-quality reference data
Human evaluation	People score with a rubric	Nuance, domain judgment, safety	Slow and expensive
LLM-as-a-judge	Another model scores against a rubric	Open-ended relevance, coherence, groundedness	Can be biased/inconsistent; calibrate it
Production signals	Measure real usage/outcomes	User value, reliability, business impact	Noisy and affected by many factors

9.1 Prefer deterministic checks when possible

If a requirement can be checked with code, do not replace it with a subjective model judge. Example: validate JSON with a schema, verify a tool parameter directly, and assert a database side effect with a test.

9.2 Use LLM judges for semantic dimensions

> **Example judge rubric**
>
> Score answer relevance from 1 to 5. A score of 5 directly answers every part of the question with no meaningful digression. A score of 1 fails to address the user's intent.

Calibrate an LLM judge against a smaller human-labeled set. The goal is not to assume the judge is perfect; the goal is to make the automated score sufficiently aligned with trusted human decisions.

10. Building an Evaluation Dataset

A good metric is useless if the test cases do not represent the real task

10.1 Include multiple case types

Case type	Example
Happy path	Clear policy question with one obvious source
Ambiguous	“Can I take leave immediately?” without specifying leave type
Multi-hop	Answer requires leave policy + employee profile
No-answer / insufficient evidence	Policy does not mention requested condition
Adversarial	Document contains prompt injection or misleading instructions
Formatting constraint	Return strict JSON / structured output
Tool failure	HR API times out or returns an error
Permission boundary	User asks for another employee’s private data

10.2 Suggested test-case schema

Field	Purpose
id	Stable test identifier
input	User query / task
expected_answer	Optional reference answer
expected_evidence	Gold document/chunk IDs for RAG
expected_tools	Allowed/expected tool calls for agents
constraints	Format, policy, role, or safety requirements
tags	Domain, difficulty, failure type, language, etc.

11. End-to-End Evaluation Scorecard

Combine metrics without hiding critical failures

11.1 Example release scorecard

Area	Metric	Target	Weight / Gate
LLM	Answer correctness	>= 90%	20%
LLM	Instruction following	>= 98%	Gate for structured workflows
RAG retrieval	Recall@5	>= 92%	15%
RAG generation	Faithfulness	>= 95%	20%
Agent	Task success	>= 95%	20%
Agent	Tool argument accuracy	>= 98%	Gate for side-effect tools
Production	p95 latency	<= 8 s	10%
Production	Cost / successful task	<= target budget	10%
Safety	Critical policy violations	0	Hard gate

Gate vs weighted metric

Do not let a high relevance score compensate for a critical safety violation. Some requirements should be hard release gates, while others can be combined into a weighted quality score.

11.2 Diagnose by layer

Observed failure	Likely place to investigate first
Answer is wrong because source was never retrieved	Retrieval recall / ranking
Source was retrieved but answer invents extra facts	Faithfulness / prompt / generation
Agent uses wrong API	Tool selection / tool descriptions / planning
Correct workflow but too slow	Latency breakdown and sequential calls
Good offline score but users keep rephrasing	Dataset mismatch, answer relevance, UX

12. Practical Evaluation Playbook

A repeatable process for development, CI/CD, and production

12.1 During development

- Start with 30-100 representative test cases before chasing large benchmark numbers.
- Record a baseline before changing prompts, models, chunking, retrievers, or agent tools.
- Store traces: model output, retrieved chunks, tool calls, tool outputs, token usage, and timings.
- Inspect failures individually; averages hide systematic failure modes.

12.2 In CI/CD

- Run a small deterministic regression suite on every important change.
- Run a broader semantic/LLM-judge evaluation before release.
- Block deployment on hard gates such as schema failures, unsafe tool actions, or severe quality regression.
- Compare the candidate against the current production baseline, not only against an arbitrary target.

12.3 In production

- Monitor latency, errors, token usage, cost, and user feedback continuously.
- Sample real traces for quality evaluation while respecting privacy and retention requirements.
- Turn real failures into new offline test cases.
- Re-evaluate when models, prompts, retrieval indexes, policies, or tools change.

The evaluation flywheel

Production failure -> add test case -> reproduce offline -> fix -> regression test -> deploy -> monitor again.

13. Common Evaluation Mistakes

Avoid the patterns that make evaluation numbers misleading

Mistake	Why it is a problem	Better approach
Using one overall score	Hides the component causing failure	Keep layer-specific metrics
Only testing happy paths	Inflates apparent quality	Include ambiguous, no-answer, adversarial, and failure cases
Using accuracy for every task	Open-ended tasks may have multiple valid outputs	Use rubrics / semantic evaluation when needed
Judging RAG only by final answer	Cannot distinguish retrieval from generation errors	Evaluate retrieval and generation separately
Optimizing cost before success	Can create a cheap but unusable system	Protect quality/safety gates first
Trusting LLM judge blindly	Judge may have bias or drift	Calibrate with human labels and deterministic checks
Ignoring percentile latency	Average hides slow-tail user experience	Track p50, p95, p99

14. Cheat Sheet

Which metric should I use?

If you want to know...	Use...
Is the answer factually right?	Correctness / accuracy
Did it answer the actual question?	Answer relevance / relevance
Did it invent unsupported claims?	Hallucination / faithfulness
Did the retriever return mostly useful chunks?	Precision@k
Did the retriever find all important evidence?	Recall@k
Did any useful result appear?	Hit Rate@k
How early did the first useful result appear?	MRR
Are highly relevant results ranked near the top?	NDCG@k
Did the agent finish the requested task?	Task success
Did the agent choose/call tools correctly?	Tool selection + tool call accuracy
Is the workflow efficient?	Steps + cost + latency
Is the live app dependable?	Reliability + production telemetry
Do real users find it useful?	User feedback / business outcome

14.1 Final mental model

GenAI Evaluation Stack

LLM quality -> RAG evidence quality -> Agent action quality -> Production system quality. Evaluate each layer independently, then combine them into an end-to-end release decision.

Good evaluation is not a single metric. It is a test system that tells you what failed, why it failed, and whether the next version is actually better.